تنافسي البرامجيات

جامعة واسط - كلية علوم الحاسوب وتكنولوجيا المعلومات

https://t.me/cs_mscX

ملخص جابتر 6

القوائم والمؤشرات وتهجينها

List and Iterator ADTs

من المصدر:

Data Structures & Algorithms in C++, 2nd, ed Michael T. Goodrich

إعداد: قناة تنافسي البرامجيات

المتجهات (Vectors)

🎯 المفهوم الأساسي

تعريف Vector

Vector = المصفوفة الذكية / المصفوفة المطاطية

الاسم البديل: Array List (قائمة المصفوفات)

المبدأ الأساسي

تسلسل يدعم الوصول المباشر باستخدام الفهارس (Indices)

- الفهرس (Index): رقم يحدد موقع العنصر في التسلسل
- الفهرس الأول: **0**
- الفهرس الأخير: **n-1** (حيث n = عدد العناصر)

تشبيه بسيط 🏨

تخيل **فندق** بغرف مرقمة:

- كل غرفة لها رقم (Index)
- تقدر توصل لأي غرفة مباشرة برقمها
- ما تحتاج تدق كل الأبواب للوصول للغرفة 50!

🔄 الفرق بين Array العادي و Vector

الميزة	Array (المصفوفة العادية)	Vector (المتجه)
الحجم	ثابت (لا يتغير) ❌	ديناميكي (يتغير) ✅
التوسع	مستحيل	تلقائي ✨
الأمان	لا توجد حماية	at() يفحص الحدود ✅
السهولة	تعريف معقد	بسيط (STL)
الوصول	O(1)	O(1)

مثال المقارنة

```cpp
// Array - ثابت وجامد
int arr[5];  // 5 = حجم ثابت
arr[10];     // Undefined Behavior! ☠️

// Vector - مرن وذكي
vector<int> v;
v.push_back(10);  // يضيف ويكبر تلقائياً ✅
v.at(10);         // يرمي استثناء إذا خارج الحدود ✅
```

📋 العمليات الأساسية (Vector ADT Operations)

العملية	الوصف	التعقيد الزمني	الاستثناء
`at(i)`	إرجاع العنصر عند الفهرس i	✅ O(1)	IndexOutOfBounds
`set(i, e)`	تعديل العنصر عند i ليصبح e	✅ O(1)	IndexOutOfBounds
`insert(i, e)`	إدراج e عند الفهرس i	⚠️ O(n)	–
`erase(i)`	حذف العنصر عند الفهرس i	⚠️ O(n)	–
`size()`	عدد العناصر	O(1)	–
`empty()`	هل فارغ؟	O(1)	–

⚠️ ملاحظة مهمة جداً!

حالة استثنائية سريعة:

- `insert(n, e)` - الإضافة في **النهاية**: **O(1)** ✅
- `erase(n-1)` - الحذف من **النهاية**: **O(1)** ✅

لماذا؟ لأنه لا يوجد عناصر تحتاج للإزاحة!

جدول العمليات في STL

العملية STL	معنى	Vector ADT	التعقيد
`push_back(e)`	إضافة في النهاية	`insert(n, e)`	O(1) امتطاء
`()pop_back`	حذف من النهاية	`erase(n-1)`	O(1)
`v[i]`	وصول سريع	`at(i)`	O(1)
`v.at(i)`	وصول آمن	`at(i)`	O(1)
`()size`	عدد العناصر	`()size`	O(1)
`()empty`	هل فارغ؟	`()empty`	O(1)
`()clear`	حذف الكل	–	O(n)
`reserve(n)`	حجز سعة	–	O(n)

ميزات إضافية في STL

```cpp
vector<int> v = {10, 20, 30, 40, 50};

// الوصول للأول والأخير
cout << v.front() << endl;  // 10
cout << v.back() << endl;   // 50

// الإدراج في مكان محدد
v.insert(v.begin() + 2, 99);  // [10, 20, 99, 30, 40, 50]

// الحذف من مكان محدد
v.erase(v.begin() + 1);  // [10, 99, 30, 40, 50]

// حجز سعة مسبقاً (تحسين الأداء)
vector<int> v2;
v2.reserve(1000);  // احجز مكان لـ 1000 عنصر
// الآن push_back 1000 عنصر لن يحتاج توسع حتى!
```

الفرق بين [] و at()

```cpp
vector<int> v = {10, 20, 30};

// [] - بدون فحص (خطر)
cout << v[5];  // Undefined Behavior ☠️
// قد يطبع قيمة عشوائية
// قد يسبب crash

// at() - مع فحص (آمن)
try {
    cout << v.at(5);  // يرمي استثناء ✅
} catch (out_of_range& e) {
    cout << "Index out of bounds!" << endl;
}
```

📊 جدول المقارنة الشامل

مقارنة التنفيذات

الميزة	Array	Vector (Array-based)	Linked List
الوصول المباشر	✅ O(1)	✅ O(1)	❌ O(n)
insert في البداية	O(n)	O(n)	✅ O(1)
insert في النهاية	O(1)	✅ O(1) امتطاء	✅ O(1)
erase من البداية	O(n)	O(n)	✅ O(1)
erase من النهاية	O(1)	✅ O(1)	✅ O(1)
الذاكرة المتلاصقة	✅	✅	❌
الحجم الديناميكي	❌	✅	✅

مقارنة العمليات

العملية	أفضل حالة	أسوأ حالة	امتطاء
at(i)	O(1)	O(1)	O(1)
set(i, e)	O(1)	O(1)	O(1)
insert(i, e)	O(1) (i=n)	O(n) (i=0)	–
erase(i)	O(1) (i=n-1)	O(n) (i=0)	–
push_back	O(1)	O(n) (توسع)	✅ O(1)
pop_back	O(1)	O(1)	O(1)

📝 ملخص سريع للمراجعة

المفاهيم الأساسية

✅ **Vector**: مصفوفة ذكية بحجم ديناميكي

✅ **Index**: الفهرس من 0 إلى n-1

✅ **الوصول**: O(1) - سريع جداً

✅ **Insert/Erase في الوسط**: O(n) - بطيء (تزحيف)

✅ **Push/Pop في النهاية**: O(1) امتطاء - سريع

المتغيرات الثلاثة

```cpp
int* A;          // المصفوفة
int n;           // الحجم الحالي
int capacity;    // السعة القصوى
```

قواعد الإزاحة

```cpp
// Insert: نبدأ من الآخر ونرجع
for (int j = n-1; j >= i; j--)
    A[j+1] = A[j];

// Erase: نبدأ من الموقع ونتقدم
for (int j = i+1; j < n; j++)
    A[j-1] = A[j];
```

التوسع التلقائي

```text
if (n >= capacity)
    reserve(2 * capacity);
```

STL Vector

```cpp
push_back()  // امتطاء O(1) سريع
pop_back()   // O(1) سريع
v[i]         // سريع، خطر
v.at(i)      // سريع، آمن
```

القوائم (Lists)

🎯 المفهوم الأساسي

تعريف List

List = تسلسل من العناصر يعتمد على **الموضع (Position)** بدلاً من **الفهرس (Index)**

الفرق الجوهري: Vector vs List

```text
Vector (المصفوفة):
[10] [20] [30] [40] [50]
  ↑    ↑    ↑    ↑    ↑
  0    1    2    3    4  ← الفهارس (متجاورة بالذاكرة)

List (القائمة المتصلة):
[10]→[20]→[30]→[40]→[50]
  ↑    ↑    ↑    ↑    ↑
 p1   p2   p3   p4   p5  ← المواضع (متفرقة بالذاكرة)
```

التشبيه البشري 🧩

Vector = فندق الغرف المرقمة 🏨

- الغرف متجاورة ومرقمة (0، 1، 2، ...)
- "روح للغرفة 5" → وصول مباشر O(1)
- بس إذا تريد تضيف غرفة بالنص → لازم تزحف الكل!

List = سبحة الخرز 📿

- الخرزات مطشرة، كل وحدة متصلة بخيط
- ما عندك "رقم"، عندك "مسكة" (Iterator)
- "احذف هذا الخرز اللي ممسكه" → قطع الخيط O(1)
- بس إذا تريد توصل للخرزة الـ50 → لازم تمشي خرزة خرزة

🔗 بنية العقدة (Node Structure)

العقدة الأساسية (Node)

كل عنصر في List موجود داخل **عقدة (Node)** تحتوي:

```cpp
struct Node {
    int data;       // البيانات (الكنز)
    Node* next;     // المؤشر للعقدة التالية (الخريطة)
};
```

📋 أنواع القوائم المتصلة

1. Singly Linked List (القائمة الأحادية)

```text
[10] → [20] → [30] → [40] → NULL
 ↑
head

كل عقدة تعرف اللي بعدها فقط
```

الميزات والعيوب

الميزة	التقييم
الذاكرة	✅ أقل (مؤشر واحد فقط)
المشي للأمام	✅ سهل
المشي للخلف	❌ مستحيل
الحذف	⚠️ يحتاج العقدة السابقة

2. Doubly Linked List (القائمة المزدوجة) ⭐

```text
     ┌────┐     ┌────┐     ┌────┐
NULL←┤prev│←───┬┤prev│←───┬┤prev│
     │ 10 │    │└ 20 │    │└ 30 │
     │next├───→┘│next├───→┘│next├→NULL
     └────┘     └────┘     └────┘

       ↑
     head
```

الميزات والعيوب

الميزة	التقييم
الذاكرة	⚠️ أكثر (مؤشران)
المشي للأمام	✅ سهل
المشي للخلف	✅ سهل
الحذف	✅ لا يحتاج العقدة السابقة
الإدراج	✅ أسهل في أي مكان

🛡️ الحراس (Sentinels): السر العبقري!

المشكلة بدون Sentinels

```cpp
// حذف عقدة - معقد جداً!
void remove(Node* v) {
    if (v == head) {
        // حالة خاصة للأول ❌
        head = head->next;
    } else if (v->next == NULL) {
        // حالة خاصة للأخير ❌
        // نحتاج نلف لإيجاد السابق!
    } else {
        // الحالة العامة
        v->prev->next = v->next;
        v->next->prev = v->prev;
    }
    delete v;
}
```

الفوائد الذهبية 🎯

```cpp
// حذف عقدة - بسيط وموحد! ✅
void remove(DNode* v) {
    // كل عنصر له next و prev لا حالات خاصة!
    v->prev->next = v->next;
    v->next->prev = v->prev;
    delete v;
    n--;
}
```

لماذا Sentinels عبقرية؟

بدون Sentinels	مع Sentinels
حالة خاصة للأول ❌	لا حالات خاصة ✅
حالة خاصة للأخير ❌	كود موحد ✅
if/else كثيرة ❌	كود نظيف ✅
احتمال أخطاء أكبر ❌	أكثر أماناً ✅

👈 Iterator و Position

المفهوم الأساسي

Iterator = "إصبعك" الذي يأشر على عنصر معين

```text
List: [10] → [20] → [30] → [40]
                ↑
            iterator p
          (مأشر على 20)
```

Position vs Iterator

المفهوم	الوصف	مثال
Position	يشير لموقع ثابت	"العنصر الموجود هنا"
Iterator	Position + القدرة على التحرك	"اتحرك للتالي (++)"

لماذا Iterator بدلاً من Index؟

```cpp
// Vector - استخدام Index
vector<int> v = {10, 20, 30};
v[1] = 99;  // O(1) - وصول مباشر ✅

// List - Index بطيء!
list<int> L = {10, 20, 30};
// للوصول لـ index=1:
// 1. ابدأ من head
// 2. امش خطوة واحدة (next)
// 3. الآن وصلت!
// → O(n) في أسوأ حالة ❌

// List - استخدام Iterator
list<int>::iterator p = L.begin();
++p;  // تقدم للعنصر التالي
*p = 99;  // عدّل العنصر الحالي
// → O(1) لأنك أصلاً عند الموقع! ✅
```

📋 List ADT: العمليات الأساسية

جدول العمليات

العملية	الوصف	التعقيد	الاستخدام
`begin()`	iterator للبداية	O(1)	التنقل
`end()`	iterator وهمي بعد النهاية	O(1)	التنقل
`insertFront(e)`	إضافة في البداية	✅ O(1)	–
`insertBack(e)`	إضافة في النهاية	✅ O(1)	–
`insert(p, e)`	إضافة قبل الموقع p	✅ O(1)	–
`eraseFront()`	حذف من البداية	✅ O(1)	–
`eraseBack()`	حذف من النهاية	✅ O(1)	–
`erase(p)`	حذف من الموقع p	✅ O(1)	–
`size()`	عدد العناصر	O(1)	–
`empty()`	هل فارغة؟	O(1)	–

⭐ ملاحظة مهمة: كل العمليات O(1)!

```text
Vector:
- الوصول: O(1) ✅
- Insert في الوسط: O(n) ❌
- Erase من الوسط: O(n) ❌

List:
- الوصول بالفهرس: O(n) ❌
- Insert في أي مكان: O(1) ✅ (إذا عندك Iterator)
- Erase من أي مكان: O(1) ✅ (إذا عندك Iterator)
```

🔍 أمثلة Tracing للامتحان

مثال 1: تتبع insertBefore

```
Initial List:
header ↔ [10] ↔ [20] ↔ [30] ↔ trailer

insertBefore(p, 25) حيث p يأشر على 30:

Step 1: إنشاء عقدة جديدة
newNode = [25]

Step 2: ربط newNode والعقدة قبلها (20) بـ 30
newNode->next = [30]
newNode->prev = [20]

Step 3: تحديث روابط 20 و 30
[20]->next = [25]
[30]->prev = [25]

Result:
header ↔ [10] ↔ [20] ↔ [25] ↔ [30] ↔ trailer
```

مثال 2: تتبع erase

```
Initial List:
header ↔ [10] ↔ [20] ↔ [30] ↔ trailer

                                                erase(p) حيث p يأشر على 20:

Step 1: احصل على الجيران
v->prev = [10]
v->next = [30]

Step 2: اربط الجيران ببعض (تجاوز 20)
[10]->next = [30]
[30]->prev = [10]

Step 3: احذف العقدة
delete [20]
n--

Result:
header ↔ [10] ↔ [30] ↔ trailer
```

مثال 3: List فارغة مع Sentinels

```
List L;  // قائمة جديدة

Initial State:
header ↔ trailer
  ↑          ↑
 prev       next

insertFront(10):
header ↔ [10] ↔ trailer

insertBack(20):
header ↔ [10] ↔ [20] ↔ trailer

eraseFront():
header ↔ [20] ↔ trailer

eraseBack():
header ↔ trailer  ← فارغة مرة أخرى!

لاحظ: header و trailer دائماً موجودان! ✅
```

📊 المقارنة الشاملة: Vector vs List

جدول التعقيد الزمني

العملية	Vector	List (Doubly)	الأفضل
الوصول بالفهرس	✅ O(1)	❌ O(n)	Vector
الوصول بـ Iterator	O(1)	O(1)	متساوي
البحث عن عنصر	O(n)	O(n)	متساوي
Insert في البداية	❌ O(n)	✅ O(1)	List
Insert في النهاية	O(1) امتطاء	O(1)	متساوي
Insert في الوسط	❌ O(n)	✅ O(1)*	List
Erase من البداية	❌ O(n)	✅ O(1)	List
Erase من النهاية	O(1)	O(1)	متساوي
Erase من الوسط	❌ O(n)	✅ O(1)*	List

* بشرط أن يكون لديك Iterator للموقع

مقارنة الذاكرة

```cpp
// Vector
class ArrayVector {
    int* A;         // 8 bytes (pointer)
    int n;          // 4 bytes
    int capacity;   // 4 bytes
    // + الذاكرة للمصفوفة نفسها
};

// List Node
struct DNode {
    int data;       // 4 bytes
    DNode* next;    // 8 bytes
    DNode* prev;    // 8 bytes
    // = 20 bytes لكل عنصر!
};

النتيجة:
Vector: ذاكرة أقل، العناصر متلاصقة (cache-friendly) ✅
List: ذاكرة أكثر (overhead للمؤشرات) ❌
```

متى نستخدم كل واحد؟

استخدم Vector عندما: ✅

```cpp
// 1. وصول عشوائي متكرر
for (int i = 0; i < v.size(); i++) {
    cout << v[i];  // O(1) - سريع!
}

// 2. معظم العمليات في النهاية
v.push_back(10);  // O(1)
v.pop_back();     // O(1)

// 3. الذاكرة مهمة
// Vector أكفأ في استخدام الذاكرة
```

استخدم List عندما: ✅

```cpp
// 1. إدراج/حذف متكرر في البداية أو الوسط
L.insertFront(10);  // O(1) - سريع!

// 2. عندما تحتاج تعديلات كثيرة
list<int>::iterator it = L.begin();
while (condition) {
    if (needToRemove) {
        it = L.erase(it);  // O(1)
    } else {
        ++it;
    }
}

// 3. لا تحتاج وصول عشوائي
// (تمشي تسلسلياً فقط)
```

التسلسلات (Sequences)

🎯 المفهوم الأساسي

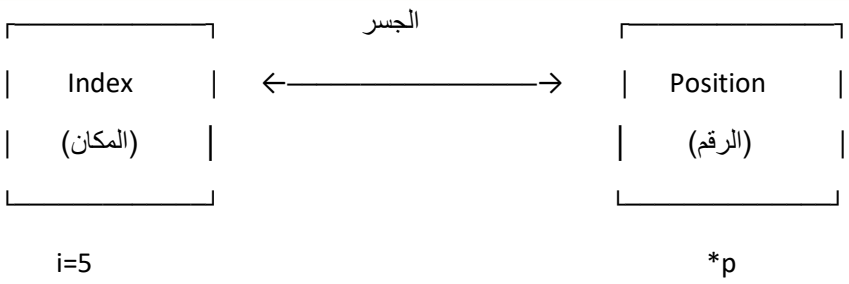
تعريف Sequence

Sequence = نوع بيانات مجرد (ADT) يجمع مميزات **Vector** و **List** معاً

🧰 المفهوم الجوهري: الجسور (Bridging Functions)

ما هي الجسور؟

Bridging Functions = دوال تربط بين عالم الفهارس وعالم المواضع



الدالتان الأساسيتان

1. **atIndex(i)** - من رقم إلى مكان 🔢→📍

```cpp
Position atIndex(int i);

// الاستخدام
Position p = seq.atIndex(5);  // أعطني Position للعنصر رقم 5
```

مثال بشري:

```text
"يا كمبيوتر، العنصر رقم 5، وين مكانه بالضبط؟"
→ يرجعلك مؤشر (Position) يأشر عليه
```

2. indexOf(p) - من مكان إلى رقم 📍→🔢

```cpp
int indexOf(Position p);

// الاستخدام
int i = seq.indexOf(p);  // هذا المكان، شقد رقمه؟
```

مثال بشري:

```text
"يا كمبيوتر، هذا العنصر اللي ممسكه بيدي، شقد رقمه بالسرة؟"
→ يرجعلك الفهرس (Index) مثل 5
```

📋 Sequence ADT: العمليات الكاملة

الواجهة الشاملة

```cpp
class Sequence {
public:
    // ═══════════ من Vector ADT ═══════════
    int size() const;                    // الحجم
    bool empty() const;                  // هل فارغ؟
    Element& at(int i);                  // الوصول بالفهرس
    void set(int i, Element e);          // التعديل بالفهرس

    // ═══════════ من List ADT ═══════════
    Iterator begin();                    // أول موقع
    Iterator end();                      // موقع وهمي بعد الآخر
    void insertFront(Element e);         // إضافة في البداية
    void insertBack(Element e);          // إضافة في النهاية
    void insert(Iterator p, Element e);  // إضافة قبل p
    void eraseFront();                   // حذف من البداية
    void eraseBack();                    // حذف من النهاية
    void erase(Iterator p);              // حذف من p

    // ═══════════ الجسور (NEW!) ═══════════
    Iterator atIndex(int i);             // 🧰 من رقم → مكان
    int indexOf(Iterator p);             // 🧰 من مكان → رقم
};
```

مثال استخدام الجسور

```cpp
Sequence<int> seq;
seq.insertBack(10);
seq.insertBack(20);
seq.insertBack(30);
seq.insertBack(40);

// استخدام :1 الطريقة Index (من Vector)
cout << seq.at(2);  // 30

// استخدام :2 الطريقة Iterator (من List)
auto it = seq.begin();
++it; ++it;
cout << *it;  // 30

// (الجديد!) استخدام الجسر :3 الطريقة
auto p = seq.atIndex(2);  // Position حول الرقم 2 لـ
cout << *p;  // 30

int index = seq.indexOf(p);  // لرقم Position حول
cout << index;  // 2
```

⚖️ التنفيذ: المعضلة الأبدية!

لا يوجد حل مثالي! ❌✅

```text
Sequence = واجهة (Interface)
تحت الغطاء: لازم نختار تنفيذ

1 الخيار: Linked List
2 الخيار: Array (Circular)

💰 !كل واحد له ثمن
```

التنفيذ الأول: Doubly Linked List

الهيكل

```cpp
class SequenceList {
private:
    DNode* header;
    DNode* trailer;
    int n;

public:
    // Iterator = مؤشر لعقدة
    typedef DNode* Iterator;

    // الجسور
    Iterator atIndex(int i);
    int indexOf(Iterator p);
};
```

تنفيذ atIndex (من رقم → مكان)

```cpp
Iterator atIndex(int i) {
    if (i < 0 || i >= n) {
        throw out_of_range("Invalid index");
    }

    // ⚠️ Link Hopping: المشي عبر العقد
    DNode* curr = header->next;  // ابدأ من أول عنصر حقيقي

    for (int j = 0; j < i; j++) {
        curr = curr->next;  // انتقل للتالي
    }

    return curr;  // Position رقم i للعنصر
}
```

التعقيد: O(n) ❌

📦 التنفيذ الثاني: Circular Array

الهيكل

```cpp
class SequenceArray {
private:
    int* A;          // المصفوفة الدائرية
    int capacity;
    int n;
    int f;           // front index

public:
    // Iterator = فهرس
    typedef int Iterator;

    // الجسور
    Iterator atIndex(int i);
    int indexOf(Iterator p);
};
```

تنفيذ atIndex

```cpp
Iterator atIndex(int i) {
    if (i < 0 || i >= n) {
        throw out_of_range("Invalid index");
    }

    // التحويل المباشر! ✅
    return (f + i) % capacity;
}
```

التعقيد: O(1) ✅ سريع جداً!

تنفيذ indexOf

```cpp
int indexOf(Iterator p) {
    if (p < 0 || p >= capacity) {
        throw invalid_argument("Invalid iterator");
    }

    // حساب بسيط ✅
    int index = (p - f + capacity) % capacity;

    if (index >= n) {
        throw invalid_argument("Iterator out of range");
    }

    return index;
}
```

التعقيد: O(1) ✅ سريع جداً!

المشكلة: insert/erase في الوسط

```cpp
void insert(Iterator p, int e) {
    // نحتاج إزاحة! ❌
    int index = indexOf(p);

    // إزاحة جميع العناصر من index للنهاية
    for (int i = n; i > index; i--) {
        int pos1 = (f + i) % capacity;
        int pos2 = (f + i - 1) % capacity;
        A[pos1] = A[pos2];
    }

    // إدراج
    int actualPos = (f + index) % capacity;
    A[actualPos] = e;
    n++;
}
```

التعقيد: O(n) ❌ بطيء!

📊 جدول المقارنة الشامل في تنفيذ عمليات الـ Sequences

التعقيد الزمني

العملية	Array (Circular)	Linked List	الملاحظات
()size(), empty	O(1)	O(1)	متساوي ✅
at(i)	✅ O(1)	❌ O(n)	Array أفضل
()begin(), end	O(1)	O(1)	متساوي ✅
atIndex(i)	✅ O(1)	❌ O(n)	**الجسر 1**
indexOf(p)	✅ O(1)	❌ O(n)	**الجسر 2**
Iterator: ++p, --p	O(1)	O(1)	متساوي ✅
()insertFront	O(1)	O(1)	متساوي ✅
()insertBack	O(1)	O(1)	متساوي ✅
insert(p, e) وسط	❌ O(n)	✅ O(1)	List أفضل
()eraseFront	O(1)	O(1)	متساوي ✅
()eraseBack	O(1)	O(1)	متساوي ✅
erase(p) وسط	❌ O(n)	✅ O(1)	List أفضل

تنفيذ المصفوفات:

- ✅ الجسور (atIndex, indexOf): O(1) سريعة
- ❌ العمليات الاضافة والمسح في الوسط والبداية: O(n) بطيئة

التنفيذ بالقائمة المتصلة:

- ❌ الجسور (atIndex, indexOf): O(n) بطيئة
- ✅ الاضافة والحذف في الوسط O(1): سريعة

القاعدة الذهبية:

لا يمكن الحصول على كل شيء⚖️ !

المقارنة البصرية

```text
Using atIndex (Linked List):
[10, 20, 30, 40]
 ↓
atIndex(0) → start from head, walk 0 steps
atIndex(1) → start from head, walk 1 step  ← إعادة!
atIndex(2) → start from head, walk 2 steps ← إعادة!
atIndex(3) → start from head, walk 3 steps ← إعادة!

Total: 0+1+2+3 = 6 خطوات في iteration واحدة ❌

Using Iterator:
[10, 20, 30, 40]
 ↓
it = begin()  → نقطة البداية
++it          → خطوة واحدة O(1)
++it          → خطوة واحدة O(1)
++it          → خطوة واحدة O(1)

Total: 3 خطوات في iteration واحدة ✅
```

متى تستخدم ماذا؟

استخدم **Array-Based Sequence** عندما:

```cpp
✅ وصول عشوائي متكرر (seq.at(i))
✅ استخدام كثير للجسور (atIndex, indexOf)
✅ معظم العمليات في الأطراف
✅ الحجم شبه ثابت

// مثال
for (int i = 0; i < seq.size(); i++) {
    cout << seq.at(i);  // وصول مباشر سريع
}
```

استخدم List-Based Sequence عندما:

```cpp
✅ إدراج/حذف متكرر في الوسط
✅ التنقل التسلسلي فقط (Iterator)
✅ لا حاجة للوصول العشوائي
✅ حجم متغير جداً

// مثال
auto it = seq.begin();
while (it != seq.end()) {
    if (condition) {
        it = seq.erase(it);  // حذف O(1)
    } else {
        ++it;
    }
}
```

📝 ملخص سريع للمراجعة

المفاهيم الأساسية

✅ **Sequence**: يجمع Vector + List

✅ **Bridging Functions**: atIndex و indexOf

✅ **atIndex**: من رقم → مكان (Index → Position)

✅ **indexOf**: من مكان → رقم (Position → Index)

✅ **Trade-off**: لا حل مثالي!

التنفيذات

```text
Array-Based:
✅ atIndex, indexOf: O(1)
❌ insert/erase وسط: O(n)

List-Based:
❌ atIndex, indexOf: O(n)
✅ insert/erase وسط: O(1)
```

القاعدة الذهبية

```text
❌ لا تستخدم atIndex مع حلقة في Linked List
✅ استخدم Iterator دائماً للمشي التسلسلي
```

Bubble Sort

```text
❌ الطريقة الخطأ: atIndex في حلقة → O(n³) أو O(n⁴)
✅ الطريقة الصحيحة: Iterator → O(n²)
```

أسئلة Tracing

سؤال 1: تتبع العمليات

```text
SequenceList seq;
seq.insertBack(10);
seq.insertBack(20);
seq.insertBack(30);
auto p = seq.atIndex(1);
int idx = seq.indexOf(p);

ما قيمة idx؟
```

الحل:

```text
insertBack(10): [10]
insertBack(20): [10, 20]
insertBack(30): [10, 20, 30]
atIndex(1): Position 20 → 1 للعنصر رقم
indexOf(p): Position 20 لـ → Index = 1

Answer: idx = 1 ✅
```

سؤال 2: تحليل الكود

```cpp
for (int i = 0; i < seq.size(); i++) {
    auto p = seq.atIndex(i);
    cout << *p << " ";
}

ما التعقيد الزمني إذا seq هو Linked List؟
```

الحل:

```text
الحلقة: n iterations
كل iteration: atIndex(i) → O(n)

Total: O(n) × O(n) = O(n²) ❌

الطريقة الصحيحة:
auto it = seq.begin();
while (it != seq.end()) {
    cout << *it << " ";
    ++it;  // O(1)
}
→ O(n) ✅
```

جدول المقارنة الثلاثي

الميزة	Vector	List	Sequence
الوصول	Index فقط	Position فقط	**كلاهما ✨**
at(i)	✅ O(1)	غير موجود	Array: O(1), List: O(n)
Iterator	موجود	موجود	موجود
atIndex	غير موجود	غير موجود	**موجود ✨**
indexOf	غير موجود	غير موجود	**موجود ✨**
insert وسط	O(n)	O(1)	Array: O(n), List: O(1)
الاستخدام	متخصص	متخصص	**شامل ✨**